RUST Live-Coding Cheatsheet

HackerRank-style interviews · syntax, std patterns, I/O, concurrency & JSON · one A3 sheet

syntax key `keyword` `i64` `Type` `"str"` `macro!` `// comment`

★ use constantly · **E0xxx** compiler error · ▌ trap / tip

I/O 01–03 · Syntax 04–07 · Collections 08–15 · Types & Errors 16–18 · Concurrency 19–20 · JSON 21 · Algorithms 22–23

sections numbered **01**→ in reading order, column by column

01 Fast I/O

read all stdin at once — I/O

```rust
use std::io::{self, Read, Write, BufRead};
fn main() {
    let mut s = String::new();
    io::stdin().read_to_string(&mut s).unwrap();
    let mut it = s.split_ascii_whitespace().map(|x| x.parse::<i64>().unwrap());
    let n = it.next().unwrap() as usize;
    let v: Vec<i64> = it.by_ref().take(n).collect();
    // buffered output — much faster than println! inside loops ★
    let out = io::stdout(); let mut w = io::BufWriter::new(out.lock());
    writeln!(w, "{} {}", n, v[0]).unwrap();
}
```

02 HackerRank Input Patterns

line-based stubs — I/O

```rust
// read_line KEEPS the '\n' and APPENDS → trim, and clear before reuse ★
let mut line = String::new();
io::stdin().read_line(&mut line).unwrap();
let n: usize = line.trim().parse().unwrap();    line.clear();
// helper: parse one whole line into Vec<T>
fn read_vec<T: std::str::FromStr>() -> Vec<T> where T::Err: std::fmt::Debug {
    let mut s = String::new(); io::stdin().read_line(&mut s).unwrap();
    s.split_whitespace().map(|x| x.parse().unwrap()).collect()
}
let v: Vec<i64> = read_vec();   let (a, b) = (v[0], v[1]);   // "a b"
// line iterator: header "n m", then n rows (int matrix / char grid)
let stdin = io::stdin();
let mut lines = stdin.lock().lines().map(|l| l.unwrap());
let hdr: Vec<usize> = lines.next().unwrap()
    .split_whitespace().map(|x| x.parse().unwrap()).collect();
let (n, m) = (hdr[0], hdr[1]);
let grid: Vec<Vec<i32>> = (0..n).map(|_| lines.next().unwrap()
    .split_whitespace().map(|x| x.parse().unwrap()).collect()).collect();
let g: Vec<Vec<u8>> = (0..n)                    // "#..#" rows -> bytes
    .map(|_| lines.next().unwrap().trim().as_bytes().to_vec()).collect();
// q command queries: "push 5" / "pop" — slice patterns on the tokens
for _ in 0..q {
    let l = lines.next().unwrap();  let t: Vec<_> = l.split_whitespace().collect();
    match t[..] { ["push", x] => st.push(x.parse().unwrap()),
                  ["pop"] => { st.pop(); }  _ => {} } }
for l in lines { if l.trim().is_empty() { continue; } }   // until EOF
// output: space-joined row · fixed decimals · YES/NO
let s = v.iter().map(|x| x.to_string()).collect::<Vec<_>>().join(" ");
println!("{:.6}", ans);   println!("{}", if ok { "YES" } else { "NO" });
```

Traps Windows input ends in `\r\n` → use `trim()` . Sums beyond 2^{31} → `i64` . Missing last line → `unwrap_or_default()` .

03 format! · Printing

I/O

```rust
println!("{} {}", a, b);   println!("{a} {b}");   // inline args
println!("{:?}", v);       println!("{:#?}", v);  // debug / pretty debug
println!("{:5}|{:<5}|{:^5}|{:>5}", x, x, x, x);   // width / align
println!("{:05} {:.3} {:8.2} {:+}", n, f, f, n);  // pad, precision, sign
println!("{:b} {:o} {:x} {:#X} {:e}", n, n, n, n, f);
println!("{:>w$}", x, w = 8);   eprintln!("stderr");   dbg!(x); // returns x
let s = format!("{}-{}", a, b);  use std::fmt::Write; write!(s, "{x}").unwrap();
```

04 Variables · Types · Casting

SYNTAX

```rust
let x = 5;              // immutable, i32 by default
let mut y: u64 = 10;    let z = 3.0_f64;   let big = 1u64 << 40;
const MAX: usize = 1_000_000;   const MOD: i64 = 1_000_000_007;
let (a, b) = (1, "hi"); // tuple destructure;   let x = x + 1; // shadowing
let i = y as i32;       // numeric cast (truncates, never panics)
let idx = n as usize;   // indexes MUST be usize ★
let c = b'a';  let ch = (c + 1) as char;  let d = (ch as u8 - b'0') as usize;
let k: i32 = "42".parse().unwrap();     let t = u8::try_from(300i32); // Err
i32::MAX  i64::MIN  usize::MAX  f64::INFINITY  f64::EPSILON
x.abs() x.pow(3) x.min(y) x.max(y) x.signum() a.abs_diff(b) x.clamp(lo, hi)
x.rem_euclid(m)   // non-negative modulo ★     (a + b - 1) / b  // ceil div
x.checked_add(y)  /*Option*/  x.wrapping_mul(y)  x.saturating_sub(y)
f.sqrt() f.floor() f.ceil() f.round() f.powi(2) f.powf(0.5) (x as f64).log2()
```

Trap `usize - usize` underflow panics in debug. Compute in `i64` then cast, or use `abs_diff` / `saturating_sub` .

05 Control Flow

SYNTAX

```rust
if x > 0 { } else if x < 0 { } else { }
let s = if x > 0 { "pos" } else { "neg" };   // if is an expression
for i in 0..n { }  /* 0..=n-1 */   for i in (0..n).rev() { }   // reverse
for i in 1..=n { } /* inclusive */  for i in (0..n).step_by(2) { }
for i in 1..=10 { }                 // 1..10 inclusive ★  (1..10 stops at 9)
for (i, v) in arr.iter().enumerate() { }
for v in &vec { }  /* borrow ★ */    for v in &mut vec { *v += 1; }
while let Some(x) = stack.pop() { }
let r = loop { break 42; };                  // loop returns a value
'outer: for i in 0..n { for j in 0..m { if bad { continue 'outer; } } }
```

06 match · Pattern Matching

SYNTAX

```rust
match x {
    0 => "zero",   1 | 2 => "small",   3..=9 => "mid",
    n if n < 0 => "neg",               // guard
    _ => "big",
}
match (a, b) { (0, _) => .., (_, 0) => .., _ => .. }
match opt { Some(v) if v > 3 => .., Some(v) => .., None => .. }
match s.as_str() { "add" => .., "sub" => .., _ => .. }
match slice { [] => .., [x] => .., [first, .., last] => .., [h, rest @ ..] => .. }
if let Some(v) = map.get(&k) { }
let Some(v) = opt else { return; };   // let-else ★
let Point { x, y } = p;   let ok = matches!(x, Some(1..=5));
```

07 Functions · Closures · Generics

SYNTAX

```rust
fn add(a: i32, b: i32) -> i32 { a + b }       // last expr = return value
fn sum(v: &[i64]) -> i64 { v.iter().sum() }   // take slices ★
fn push(v: &mut Vec<i32>) { v.push(1); }
fn big<T: Ord + Copy>(a: T, b: T) -> T { if a > b { a } else { b } }
fn f<T>(x: T) where T: Clone + std::fmt::Debug { }
let sq = |x: i32| x * x;    let add = |a, b| a + b;   // types inferred
let mut cnt = 0; let mut inc = || cnt += 1;   // mutates capture -> mut
let f = move |x| x + offset;                  // take ownership
fn make() -> impl Fn(i32) -> i32 { |x| x + 1 }
let boxed: Box<dyn Fn(i32) -> i32> = Box::new(sq);
// closures can't recurse -> use a fn (nested fn is OK)
fn dfs(u: usize, g: &[Vec<usize>], vis: &mut [bool]) {
    vis[u] = true; for &v in &g[u] { if !vis[v] { dfs(v, g, vis); } }
}
```

08 Sorting · Comparing

COLLECTIONS

```rust
v.sort();  v.sort_unstable();  v.sort_by(|a, b| b.cmp(a));   // descending
v.sort_by_key(|p| p.1);     v.sort_by_key(|&x| Reverse(x));
v.sort_by(|a, b| b.1.cmp(&a.1).then(a.0.cmp(&b.0)));   // score desc, name asc ★
v.sort_by_key(|p| (Reverse(p.1), p.0.clone()));        // same, as key tuple
f.sort_by(|a, b| a.total_cmp(b));      // f64 sort ★
let mut idx: Vec<usize> = (0..n).collect();   idx.sort_by_key(|&i| a[i]); // argsort
v.select_nth_unstable(k);              // k-th smallest, O(n)
a.cmp(&b) -> Ordering::{Less, Equal, Greater}   .reverse()   .then_with(|| ..)
"abc" < "abd"   (1, 'a') < (1, 'b')   vec![1, 2] < vec![1, 3]   // lexicographic
```

09 HashMap · HashSet · BTreeMap

COLLECTIONS

```rust
use std::collections::{HashMap, HashSet, BTreeMap, BTreeSet};
let mut m: HashMap<String, i32> = HashMap::new();
m.insert(k, v) /*old: Option*/   m.get(&k) /*Option<&V>*/   m[&k] /*panics*/
m.get_mut(&k)  m.contains_key(&k)  m.remove(&k)  m.len()
*m.entry(k).or_insert(0) += 1;                // counting ★
m.entry(k).or_default().push(x);              // adjacency list ★
m.entry(k).and_modify(|v| *v += 1).or_insert(1);
for (k, v) in &m { }    for (k, v) in m.iter_mut() { *v = 0; }
m.keys()  m.values()  m.values_mut()   let mut kv: Vec<_> = m.iter().collect();
let m: HashMap<_, _> = v.iter().map(|x| (x.id, x)).collect();
let mut s = HashSet::new();   s.insert(x) /* bool */  s.contains(&x)  s.remove(&x)
s.union(&t)  s.intersection(&t)  s.difference(&t)       // iterators
// BTreeMap / BTreeSet: sorted keys, range queries O(log n)
b.first_key_value()  b.last_key_value()  b.pop_first()  b.pop_last()
b.range(k..).next() /* lower_bound */   b.range(..=k).next_back() /* <= k */
```

Keys HashMap: `Eq` + `Hash` ; BTreeMap: `Ord` . `f64` can't be a key; tuples / `Vec` / `String` can.

10 Iterators: Adapters · Consumers · fold

자주 쓰는 것 — COLLECTIONS

```rust
// 어댑터 Adapters — lazy: 새 iterator 반환, 끝에 소비자가 있어야 실행됨
  .map(|x| x * 2)                          map —  각 원소 변환
  .filter(|&&x| x > 0)                     filter — 조건 맞는 것만
  .filter_map(|t| t.parse::<i32>().ok())   filter_map — 파싱 / Option 걸러내기
  .enumerate() // (i, x)                   enumerate — 인덱스가 필요할 때
  a.iter().zip(&b) // (x, y)               zip — 두 시퀀스를 같이 순회
  .cloned() .copied()                      cloned / copied — 참조(&T) → 값(T)
  .take(n) .skip(n) .step_by(k)            일부만 쓸 때
  .rev() .chain(b) .flat_map(f) .flatten()  뒤집기 · 잇기 · 펼치기
  .take_while(p) .skip_while(p) .peekable()  조건까지 · 미리보기
  .scan(0, |s, &x| { *s += x; Some(*s) })   누적 중간값 (prefix sum)
// 소비자 Consumers — eager: 값 하나 반환, 여기서 실제로 실행
  .sum::<i64>() .count() .product()        합계, 개수, 곱
  .any(|&x| x < 0) .all(|&x| x > 0)        조건 검사 → bool
  .find(|&&x| x > 10) .position(|&x| x == t)   첫 일치 → Option
  .min() .max() .max_by_key(|p| p.1)       최소 / 최대 → Option
  .collect::<Vec<_>>() .unzip() .partition(p)   모으기 / 둘로 나누기
  .last() .nth(k) .for_each(|x| ..)        마지막 · k번째 · 부수효과
// Id — 모든 소비자의 일반형:    fold(init, |acc, x| new_acc)
v.iter().fold(0, |acc, &x| acc + x)                    // = sum
v.iter().fold(i64::MIN, |m, &x| m.max(x))              // = max
v.iter().fold((0, 0), |(e, o), &x| if x % 2 == 0 {(e+1, o)} else {(e, o+1)})
s.bytes().fold(0u64, |h, b| (h * 31 + b as u64) % MOD) // rolling hash
v.iter().try_fold(0i32, |acc, &x| acc.checked_add(x)) // None on overflow
v.into_iter().reduce(|a, b| a.max(b))                  // no init -> Option
// 라이브 코딩 관용구 Live-coding idioms
let (i, &mx) = v.iter().enumerate().max_by_key(|&(_, &x)| x).unwrap(); // argmax
let ans: i64 = s.split_whitespace().filter_map(|t| t.parse::<i64>().ok())
    .filter(|&x| x % 2 == 0).map(|x| x * x).sum();   // adapters... -> consumer
let freq = s.chars().fold(HashMap::new(), |mut m, c| {
    *m.entry(c).or_insert(0) += 1; m });                // char counts
let inc = v.windows(2).all(|w| w[0] <= w[1]);           // sorted?
let diffs: Vec<i64> = v.windows(2).map(|w| w[1] - w[0]).collect();
let pairs = (0..n).flat_map(|i| (i + 1..n).map(move |j| (i, j)));  // all i<j
let rows = grid.iter().map(|r| r.iter().map(|x| x.to_string())
    .collect::<Vec<_>>().join(" ")).collect::<Vec<_>>().join("\n");
let mut it = v.iter().peekable();                       // run-length grouping
while let Some(&x) = it.next() { let mut c = 1;
    while it.next_if_eq(&&x).is_some() { c += 1; } }
let total: i64 = v.iter().map(|&x| x as i64).sum();  // widen before summing ★
```

11 iter · iter_mut · into_iter · Sources

빌림 vs 소유 — COLLECTIONS

```rust
  v.iter() for x in &v                     &T — 읽기만, v 계속 사용 가능
  v.iter_mut() for x in &mut v             &mut T — 제자리  수정 (*x += 1)
  v.into_iter() for x in v                 T — 소유권 이동, 이후 v 사용 불가
// into_iter: 값을 꺼내 새 컬렉션으로 옮길 때 (clone 없음) ★
let names = vec!["apple".to_string(), "pear".to_string(), "kiwi".to_string()];
let prices = vec![100, 200, 150];
let result: Vec<(String, i32)> = names.into_iter().zip(prices).collect();
// [("apple", 100), ("pear", 200), ("kiwi", 150)]  names·prices 이동됨
// 보물   지하려면   : names.iter().zip(&prices) -> Vec<(&String, &i32)>
let (ns, ps): (Vec<String>, Vec<i32>) = result.into_iter().unzip(); // 분리
let m: HashMap<String, i32> = ns.into_iter().zip(ps).collect();       // -> map
for (name, price) in &m { }          // &m: (&K, &V) 빌림
for (name, price) in m { }           // HashMap   into_iter: (K, V) 소유
```

규칙 `zip(b)` · `extend(b)` · `for x in b` 는 `b.into_iter()` 를 자동 호출 → `Vec` 을 넘기면 이동, `&v` 를 넘기면 빌림. (`&v`).into_iter() = v.iter() . 배열 `[T; N]` 도 2021부터 값으로 순회.

Sources · collect

```rust
(0..n)  (0..n).rev()  (a..=b).step_by(2)  s.chars()  s.bytes()  m.iter()
.collect::<String>()  ::<HashSet<_>>()  ::<HashMap<_,_>>()  ::<Result<Vec<_>,_>>()
std::iter::repeat(0).take(n)   std::iter::once(x)   std::iter::from_fn(|| ..)
std::iter::successors(Some(1), |&x| Some(x * 2))     // 1, 2, 4, 8, ...
```

Tips `|&x|` strips one reference (`|&&x|` in filter on `.iter()`). An adapter chain with no consumer does nothing — the compiler warns unused `must be used` .

12 Vec · Slices

COLLECTIONS

```rust
let mut v: Vec<i32> = Vec::new();   let v = vec![1, 2, 3];   let v = vec![0; n];
let g = vec![vec![0; m]; n];        // 2D grid n x m ★
let v: Vec<i32> = (1..=5).collect();   let v = Vec::with_capacity(n);
v.push(x)  v.pop()/*Option*/  v.insert(i, x)  v.remove(i)  v.swap_remove(i)
v.len()  v.is_empty()  v.clear()  v.truncate(k)  v.resize(n, 0)
v.first()  v.last()  v[i]  v.get(i)/*Option, safe*/  v.swap(i, j)
v.reverse()  v.contains(&x)  v.iter().position(|&e| e == x)
v.dedup()  v.retain(|&x| x > 0)  v.fill(0)  v.rotate_left(k)  v.drain(..k)
v.binary_search(&x)             // Ok(i) / Err(insert_pos)
v.partition_point(|&x| x < t)  // lower_bound ★
&v[1..3]  &v[..k]  &v[k..]  v.split_at(k)  v.windows(2)  v.chunks(3)
let (l, r) = v.split_at_mut(mid);   // two &mut halves
```

13 String · &str · char

COLLECTIONS

```rust
let s = String::from("abc");   let s = "abc".to_string();   let t: &str = &s;
let t = &s[1..3];   // BYTE index!     s.len() /* bytes */   s.chars().count()
s.push('x')  s.push_str("yz")  s.pop()  s.insert(0, 'a')  format!("{s}{t}")
s.as_bytes()  // &[u8], O(1) indexing ★
let cs: Vec<char> = s.chars().collect();   cs.iter().collect::<String>()
s.chars().rev().collect::<String>()   String::from_utf8(bytes).unwrap()
s.trim()  .to_uppercase()  .to_lowercase()  .lines()  .split(',')
s.split_whitespace()   s.splitn(2, ':')   s.split_once('=')  // Option<(&str,&str)>
s.starts_with("ab") .ends_with() .contains() .find("b")  /* Option<usize> */
s.replace("a", "b")  .repeat(3)  .is_empty()  .chars().nth(i)
s.parse::<i32>().unwrap()   n.to_string()
c.is_ascii_digit()  .is_alphabetic()  .is_alphanumeric()  .is_whitespace()
c.is_ascii_lowercase()  c.is_ascii_uppercase()  c.to_digit(10)/*Option*/
(b'a' + i as u8) as char    (c as u8 - b'a') as usize    char::from_digit(d, 10)
```

14 Array ↔ Vec · Conversion · Concat

COLLECTIONS

```rust
let a = [1, 2, 3];                  // [i32; 3]: stack, fixed length
let cnt = [0usize; 26];             // letter counts: cnt[(c - b'a') as usize] ★
let v: Vec<i32> = a.to_vec();       // array / slice -> Vec (clone)
let v = Vec::from(a);   let v: Vec<i32> = a.into();   // also into_iter().collect()
let s: &[i32] = &a;     let s = &v[..];     let s = v.as_slice();
let arr: [i32; 3] = v.try_into().unwrap();       // Vec -> array (len must match)
let arr: [i32; 3] = v[..3].try_into().unwrap();  // slice -> array (copy)
let sq: [usize; 5] = std::array::from_fn(|i| i * i);   let b = a.map(|x| x * 2);
// 이어붙이기 Concat — 원본 이동(move) vs 유지(keep)
let c = a + &b;                     // String: a 통  , b 유지
let c = format!("{a}{b}");          // String:       각  ★
a.push_str(&b);                     // String: a  붙임  , b 유지
a.extend(b);                        // Vec: a  붙임  , b 이동
a.extend_from_slice(&b);            // Vec: a  붙임  , b 유지
a.append(&mut b);                   // Vec: 소멸  임  , b      Vec
let c = [a, b].concat();            //     Vec, a·b 이동
let c = [&a[..], &b[..]].concat();  //     Vec      각  ★
let c: Vec<_> = a.iter().chain(&b).cloned().collect();   // 둘 다 유지
let flat: Vec<i32> = grid.concat(); // Vec<Vec<T>> -> Vec<T> (or .flatten())
let s = words.join(" ");            // Vec<String> / [&str] -> String
v.splice(1..1, [7, 8]); /* insert many */   let r = [1, 2].repeat(3); // 1,2,1,2,..
let s: String = chars.iter().collect();   let bytes: Vec<u8> = s.into_bytes();
```

15 VecDeque · BinaryHeap · Reverse

COLLECTIONS

```rust
use std::collections::{VecDeque, BinaryHeap};   use std::cmp::Reverse;
let mut q = VecDeque::new();        // queue / deque (BFS)
q.push_back(x)  q.push_front(x)  q.pop_front()  q.pop_back()  q.front()  q[i]
let mut h = BinaryHeap::new();      // MAX-heap
h.push(x)  h.pop()  h.peek()  h.len()  h.into_sorted_vec()
h.push(Reverse(x));  if let Some(Reverse(x)) = h.pop() { }   // min-heap
h.push(Reverse((dist, node)));      // Dijkstra ★
#[derive(PartialEq, Eq)] struct St { cost: i64, id: usize }  // custom order
impl Ord for St { fn cmp(&self, o: &Self) -> std::cmp::Ordering {
    o.cost.cmp(&self.cost).then_with(|| self.id.cmp(&o.id)) } } // reversed = min
impl PartialOrd for St { fn partial_cmp(&self, o: &Self)
    -> Option<std::cmp::Ordering> { Some(self.cmp(o)) } }
```

16 Struct · Enum · impl · Trait

TYPES & ERRORS

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Default)]
struct P { x: i32, y: i32 }     struct Pair(i32, i32);   // p.0
let p = P { x: 1, y: 2 };   let q = P { x: 5, ..p };
impl P {
    fn new(x: i32, y: i32) -> Self { Self { x, y } }     // P::new(1, 2)
    fn dist(&self) -> i32 { self.x.abs() + self.y.abs() }
    fn shift(&mut self, d: i32) { self.x += d; }
}
enum Shape { Circle(f64), Rect { w: f64, h: f64 }, Empty }
enum Dir { N, E, S, W }     Dir::N as usize   // 0
trait Area { fn area(&self) -> f64;
    fn name(&self) -> String { "?".into() } }          // default method
impl Area for P { fn area(&self) -> f64 { 0.0 } }
fn show(a: &impl Area) { }   fn show<T: Area>(a: &T) { }   fn show(a: &dyn Area) { }
let v: Vec<Box<dyn Area>> = vec![Box::new(p)];
impl std::fmt::Display for P {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "({}, {})", self.x, self.y) } }
```

17 Ownership · Borrowing · Lifetimes

TYPES & ERRORS

```rust
let a = String::from("x"); let b = a;     // a moved, unusable
let b = a.clone();  /* deep copy */       let n = 5; let m = n;  // Copy types
fn f(s: &String)                  // shared borrow (read)
fn f(s: &mut String)  // exclusive borrow (one at a time)
fn f(s: String)       // takes ownership
fn f(s: &str) / f(v: &[T])   // slices accept more callers ★
E0502 mut + shared borrow at once -> clone value first: let x = v[i]; v.push(x);
E0499 two mut borrows -> v.swap(i, j) / split_at_mut / std::mem::swap
E0382 use after move  -> iterate &v / .iter() / .clone()
E0308 type mismatch   -> `as` cast, &s vs s, usize vs i32
E0106 missing lifetime -> fn f<'a>(a: &'a str, b: &'a str) -> &'a str
                          struct W<'a> { s: &'a str }  // or just own a String
map.get(&k).copied().unwrap_or(0)          // end the borrow, get the value
std::mem::take(&mut v)    std::mem::replace(&mut a, b)    (a, b) = (b, a);
use std::{rc::Rc, cell::RefCell};       // shared mutable, single thread
let n = Rc::new(RefCell::new(0)); *n.borrow_mut() += 1; let n2 = Rc::clone(&n);
```

18 Option · Result · ? · Errors

TYPES & ERRORS

```rust
o.unwrap() o.expect("msg") o.unwrap_or(0) o.unwrap_or_default() o.is_some()
o.unwrap_or_else(|| calc())  o.map(|x| x + 1)  o.and_then(f)  o.filter(|x| ..)
o.take()  o.replace(v)  o.as_ref()  o.as_mut()  o.or(b)
o.get_or_insert(v)  o.zip(p)  o.map_or(0, |x| x)  o.is_none()
r.map_err(|e| ..)   r.unwrap_or(d)  r.unwrap_or_default()  r.is_ok()  r.is_err()
// Option <-> Result conversion ★
o.ok_or(e)   o.ok_or_else(|| e)          // Option<T> -> Result<T, E>
r.ok()       r.err()                     // Result<T, E> -> Option<T> / Option<E>
o.transpose()  r.transpose()   // Option<Result<T,E>> <-> Result<Option<T>,E>
let x = o.ok_or("missing")?;           // use ? on an Option in a Result fn
let n = s.parse::<i32>().ok()?;        // use ? on a Result in an Option fn
let v: Vec<i32> = a.iter().map(|s| s.parse()).collect::<Result<_,_>>()?; // 1st Err
let v: Option<Vec<u32>> = a.iter().map(|x| x.checked_mul(2)).collect(); // any None
let v: Vec<i32> = a.iter().filter_map(|s| s.parse().ok()).collect();   // skip bad
o.is_some_and(|x| x > 0)   o.is_none_or(|x| x > 0)   r.is_ok_and(|x| x > 0)
o?  r?                  // early-return None / Err from the fn ★
fn parse(s: &str) -> Result<i32, String> {
    let n: i32 = s.parse().map_err(|e| format!("{e}"))?;
    if n < 0 { return Err("negative".into()); }
    Ok(n)
}
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let n: i32 = "42".parse()?;    // any error type converts
    Ok(())
}
panic!("bad {x}"); unreachable!(); todo!(); assert!(c); assert_eq!(a, b);
```

19 Channels · mpsc

multi-producer, single-consumer — CONCURRENCY

```rust
use std::{sync::mpsc, thread, time::Duration};
let (tx, rx) = mpsc::channel::<i32>();   // unbounded: send never blocks
let (tx, rx) = mpsc::sync_channel(16);   // bounded: send blocks when full
tx.send(x).unwrap();                     // Err if receiver dropped
rx.recv()                                // blocks; Err once ALL tx dropped
rx.try_recv()   rx.recv_timeout(Duration::from_millis(100))   rx.try_iter()
for msg in rx { }                        // ends when every tx is dropped ★
// N producers -> 1 consumer
let (tx, rx) = mpsc::channel();
let hs: Vec<_> = (0..4).map(|id| {
    let tx = tx.clone();                 // one clone per producer ★
    thread::spawn(move || { for i in 0..3 { tx.send((id, i)).unwrap(); } })
}).collect();                            // each tx drops when its thread ends
drop(tx);                                // drop the original or `for` hangs ★
for (id, i) in rx { println!("{id}: {i}"); }
for h in hs { h.join().unwrap(); }
// scoped threads may borrow locals (no Arc, no 'static)
thread::scope(|s| { s.spawn(|| work(&data)); s.spawn(|| work(&data)); });
```

Note `Sender` is `Clone` ; `Receiver` is NOT `Clone` → exactly one consumer. Many consumers → share it via `Arc<Mutex<_>>` (see the SPMC card).

20 SPMC · Worker Pool

single producer, multiple consumers — CONCURRENCY

```rust
use std::sync::{mpsc, Arc, Mutex};
let (tx, rx) = mpsc::channel::<u64>();       // jobs
let rx = Arc::new(Mutex::new(rx));           // ONE receiver, shared ★
let (rtx, rrx) = mpsc::channel();            // results back (MPSC)
let workers: Vec<_> = (0..4).map(|id| {
    let (rx, rtx) = (Arc::clone(&rx), rtx.clone());
    thread::spawn(move || loop {
        let job = rx.lock().unwrap().recv(); // lock released at `;` ★
        match job {
            Ok(n) => rtx.send((id, n * n)).unwrap(),
            Err(_) => break,                 // channel closed -> exit
        }
    })
}).collect();
drop(rtx);                                   // only workers hold senders
for n in 0..10 { tx.send(n).unwrap(); }      // the single producer
drop(tx);                                    // close -> workers exit
let results: Vec<(usize, u64)> = rrx.iter().collect();
for w in workers { w.join().unwrap(); }
use std::sync::atomic::{AtomicU64, Ordering};      // lock-free counter
let c = Arc::new(AtomicU64::new(0));       c.fetch_add(1, Ordering::Relaxed);
let (tx, rx) = crossbeam_channel::bounded::<u64>(16); // crate: MPMC, rx: Clone
```

Trap `while let Ok(j) = rx.lock().unwrap().recv() { .. }` holds the lock for the WHOLE body → workers run one at a time. Bind with `let` first.

21 JSON · serde_json

serde (derive) + serde_json — JSON

```rust
use serde::{Deserialize, Serialize};   use serde_json::{json, Value};
#[derive(Serialize, Deserialize, Debug, Clone)]
#[serde(rename_all = "camelCase")]          // user_id <-> "userId"
struct User {
    user_id: u64,
    name: String,
    #[serde(default)] tags: Vec<String>,    // missing -> empty
    email: Option<String>,                  // null / missing -> None
    #[serde(rename = "type")] kind: String,  // reserved-word key
    #[serde(skip_serializing_if = "Option::is_none")] nick: Option<String>,
}
let u: User = serde_json::from_str(s)?;          // &str -> struct ★
let us: Vec<User> = serde_json::from_str(s)?;    // JSON array
let s = serde_json::to_string(&u)?;   let s = serde_json::to_string_pretty(&u)?;
// untyped: serde_json::Value
let v: Value = serde_json::from_str(r#"{"a":{"b":[1,2]}}"#)?;
v["a"]["b"][0]                 // missing key -> Value::Null, never panics
v["a"]["b"][0].as_i64()        // Option<i64>; as_str as_f64 as_bool as_array
v.get("a").and_then(|x| x.get("b"))    v.pointer("/a/b/1")    // Option<&Value>
let mut v = json!({ "id": 1, "tags": ["x", "y"], "ok": true, "n": null });
v["id"] = json!(2);    v["tags"].as_array_mut().unwrap().push("z".into());
let u: User = serde_json::from_value(v)?;   let v = serde_json::to_value(&u)?;
#[derive(Serialize, Deserialize)]
#[serde(tag = "type", rename_all = "snake_case")]   // {"type":"order","id":1}
enum Msg { Ping, Order { id: u64, px: f64 } }
```

22 Algorithm Templates

ALGORITHMS

```rust
// BFS on a grid
let d = [(0, 1), (1, 0), (0, -1), (-1, 0)];
let mut dist = vec![vec![-1; m]; n]; dist[sr][sc] = 0;
let mut q = VecDeque::from([(sr, sc)]);
while let Some((r, c)) = q.pop_front() {
    for &(dr, dc) in &d {
        let (nr, nc) = (r as i32 + dr, c as i32 + dc);
        if nr < 0 || nc < 0 || nr >= n as i32 || nc >= m as i32 { continue; }
        let (nr, nc) = (nr as usize, nc as usize);
        if grid[nr][nc] == b'#' || dist[nr][nc] != -1 { continue; }
        dist[nr][nc] = dist[r][c] + 1; q.push_back((nr, nc));
    }
}
// Dijkstra
let mut dist = vec![i64::MAX; n]; dist[s] = 0;
let mut pq = BinaryHeap::from([Reverse((0i64, s))]);
while let Some(Reverse((d, u))) = pq.pop() {
    if d > dist[u] { continue; }
    for &(v, w) in &adj[u] {
        let nd = d + w;
        if nd < dist[v] { dist[v] = nd; pq.push(Reverse((nd, v))); }
    }
}
// Union-Find (path compression)
fn find(p: &mut Vec<usize>, x: usize) -> usize {
    if p[x] != x { p[x] = find(p, p[x]); } p[x] }
fn union(p: &mut Vec<usize>, a: usize, b: usize) {
    let (a, b) = (find(p, a), find(p, b)); if a != b { p[a] = b; } }
// binary search on answer: smallest x with ok(x)
let (mut lo, mut hi) = (0i64, 1e18 as i64);
while lo < hi { let mid = lo + (hi - lo) / 2;
    if ok(mid) { hi = mid; } else { lo = mid + 1; } }
// DP table · prefix sum · sliding window
let mut dp = vec![vec![0i64; m + 1]; n + 1];
let mut pre = vec![0; n + 1];
for i in 0..n { pre[i + 1] = pre[i] + a[i]; }     // sum[l..r] = pre[r] - pre[l]
let (mut l, mut sum) = (0, 0);
for r in 0..n { sum += a[r]; while sum > k { sum -= a[l]; l += 1; } }
// fast modular power
fn pw(mut b: i64, mut e: i64, m: i64) -> i64 { let mut r = 1; b %= m;
    while e > 0 { if e & 1 == 1 { r = r * b % m; } b = b * b % m; e >>= 1; } r }
```

23 Misc · Easy to Forget

ALGORITHMS

```rust
fn gcd(a: u64, b: u64) -> u64 { if b == 0 { a } else { gcd(b, a % b) } }  // lcm
  node.left.as_ref().map_or(0, |n| n.val)        Option<Box<Node>> child
  x.count_ones() x.leading_zeros() x.trailing_zeros()   bits · x & (x - 1)
  1u64 << k b.pow(e)                             shift / exponent must be u32
  #![allow(dead_code, unused)]                   silence warnings (file top)
  v.len() - 1 on an empty Vec                    panics → saturating_sub(1)
  std::env::args().collect::<Vec<_>>()           CLI args
  std::fs::read_to_string(path)?                 read a whole file
```

Routine ① parse input first ② decide `i64` vs `usize` boundaries ③ signatures take `&[T]` / `&str` ④ stuck on a borrow? `.clone()` now, optimize later ⑤ `dbg!` intermediate values ⑥ `--release` when timing.